Wholesale Inventory Architecture Plan

Current State

Item	Detail
Total products in CRM	~5,000 (synced from TunerStop main via `ProductSyncService`)
Wholesale-relevant products	~500–1,000
Products shown on wholesale now	**All active products** (`status = 1`) — no distinction
Inventory system	`product_inventories` table (per-warehouse, per-variant) — data exists but **not enforced**
Cart stock validation	**None** — users can add unlimited quantity

Current Data Flow

```
TunerStop Admin → POST /api/product-sync → ProductSyncService → products + product_variants
                                                                          ↓
                                                        All active products shown on wholesale
                                                        (no wholesale filter exists)
```

Problems to Solve

1. **Cart over-quantity** — users can add 20 units when only 5 are in stock
2. **Wrong product scope** — all 5,000 TunerStop products show on wholesale; only 500–1,000 should
3. **No inventory management UI** — no Filament admin page to manage or view wholesale product stock

Options Evaluated

Option A: Separate "Inventory Assets" Section

A completely separate section/table for wholesale products, uploaded and managed independently.

Pros:

- Clean separation of wholesale vs TunerStop products
- Custom fields for wholesale-specific data

Cons:

- Duplicates product data (images, specs, pricing) that already exists
- Must be kept in sync with main products manually — maintenance burden
- Non-standard pattern — creates two sources of truth

Option B: `track_inventory` Flag on Existing Products ✅ Recommended

Add boolean flags directly to the existing `products` table. Same product record, same sync pipeline.

Pros:

- Zero data duplication
- TunerStop sync continues working unchanged
- Industry standard (Shopify, WooCommerce, Odoo, ERPNext all use this exact approach)
- Filament admin: just a toggle column on the existing product grid
- The `product_inventories` table already exists — just enforce it

Industry References:

Platform	Flag Name	Behavior
Shopify	`track_quantity` (per variant)	Enables inventory enforcement
WooCommerce	`manage_stock` checkbox	Unlocks stock qty field
Magento	`manage_stock` in Inventory	Per-product stock management toggle

Platform	Flag Name	Behavior
Odoo	`type = storable`	Storable = tracked, Consumable = untracked
ERPNext	`maintain_stock` boolean	Controls whether item participates in stock ledger

Recommended Solution: Two New Columns on `products`

```
ALTER TABLE products
  ADD COLUMN available_on_wholesale BOOLEAN DEFAULT false,
  ADD COLUMN track_inventory        BOOLEAN DEFAULT false;
```

Column Definitions

Column	Purpose
`available_on_wholesale`	Controls which products appear on tunerstopwholesale.com
`track_inventory`	Controls whether stock is enforced in cart (allows backorder/pre-order flexibility)

Why two separate flags?

- A product can be *available on wholesale* but not track inventory (e.g. a made-to-order item)
- A product can track inventory but not be listed on wholesale (internal use)
- Separating them gives full flexibility

Implementation Phases

Phase 1 — Cart Quantity Validation (Immediate)

Enforce existing `product_inventories.quantity` in the cart before the Product/Inventory flags are added.

Backend changes:

- `CartService::addItem()` — check total inventory across warehouses before adding
- `CartService::changeQuantity()` — check inventory before increasing qty
- Return a clear error message: `"Only X units available in stock"`

Frontend changes:

- `ShoppingCartComponent` — show error toastr when backend rejects over-quantity
- Cap the `+` button based on available stock returned in the cart response

Phase 2 — Product Flags + Filament Admin

1. **Migration** — add `available_on_wholesale` + `track_inventory` to `products` table
2. **Product model** — add to `$fillable`, add scopes `scopeWholesale()`, `scopeTrackedInventory()`
3. **Wholesale ProductController** — filter: `WHERE available_on_wholesale = 1`
4. **New Filament `ProductResource`** — grid with:
   - All products from the main sync
   - Toggle columns for `available_on_wholesale` and `track_inventory`
   - Bulk actions: "Enable Wholesale", "Enable Inventory Tracking"
   - Filter: "Wholesale Products Only"
5. **Inventory view in Filament** — filter `product_inventories` by `track_inventory = true` products
6. **CartService** — only enforce stock limits when `product.track_inventory = true`

Target Data Flow (After Implementation)

```
TunerStop Admin → POST /api/product-sync → ProductSyncService → products + product_variants
                                                                ↓
                                                Admin toggles available_on_wholesale = true
                                                (on ~500-1000 products)
                                                                ↓
                                                Wholesale ProductController filters by available_on_w
                                                                ↓
                                                Only those products shown on tunerstopwholesale.com
                                                                ↓
                                                Cart enforces product_inventories.quantity
                                                (when track_inventory = true)
```

File Locations (Current Codebase)

File	Role
`app/Modules/Wholesale/Cart/Services/CartService.php`	Where qty validation goes (Phase 1)
`app/Http/Controllers/Wholesale/CartController.php`	Cart API endpoints
`app/Http/Controllers/Wholesale/ProductController.php`	Wholesale product listing (add wholesale filter Phase 2)
`app/Modules/Products/Models/Product.php`	Add `available_on_wholesale`, `track_inventory` to fillable
`app/Modules/Inventory/Models/ProductInventory.php`	Stock data source — `quantity` + `eta_qty` per warehouse
`app/Filament/Resources/`	New `ProductResource` goes here (Phase 2)

File	Role
`database/migrations/`	New migration for the two columns (Phase 2)
`wholesale/src/app/cart/shopping-cart/shopping-cart.component.ts`	Frontend qty cap (Phase 1)